

Department of Electronic Systems

Examination paper for TFE4141 Design of Digital System 1

Academic contact during examination: Per Gunnar Kjeldsberg

Phone: 934 59 550

Academic contact present at the exam location: YES (around 11:00 o'clock)

Examination date: Monday December 16

Examination time (from-to): 09:00 – 13:00

Permitted examination support material C: Specified printed and hand-written support material is allowed. A specific basic calculator is allowed.

You are allowed to bring the course textbook (Dally, Harting and Aamodt, "Digital design using VHDL, a systems approach") as well as four hand-written one-sided sheets of paper with your own notes.

Other information:

Read through the question set before you start answering. Plan how much time you should spend on each. If a question is unclear/vague, make your own assumptions and specify them in your answer. The total number of achievable points is 100. The maximum numbers for individual questions are indicated for each.

Language: English

Number of pages (front page excluded): 10

Number of pages enclosed: 1

Informasjon om trykking av eksamensoppgave

Originalen er:

1-sidig ☐ 2-sidig ☐

sort/hvit ☐ farger ☐

skal ha flervalgskjema ☐

Checked by:

Date

Signature

Students will find the examination results in Studentweb. Please contact the department if you have questions about your results. The Examinations Office will not be able to answer this.

Task 1 thorough 12 are Multiple Choice. Write your answer in the table in Appendix A. Detach the sheet with Appendix A and hand it in as part of your submission.

Task 1 through 6 gives a maximum of 2 point. The remaining gives a maximum of 3 points. For these last six you can include a short explanation of your answer in the regular part of your submission (not in Appendix A). If it is reasonable, you can get up to +1 point instead of -1 even if your answer in Appendix A is wrong.

Task 1: Simulation result

(Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the code in Figure 1, which timing diagram shows the correct behavior, A, B, or C?

```
library ieee;
use ieee.std_logic_1164.all;

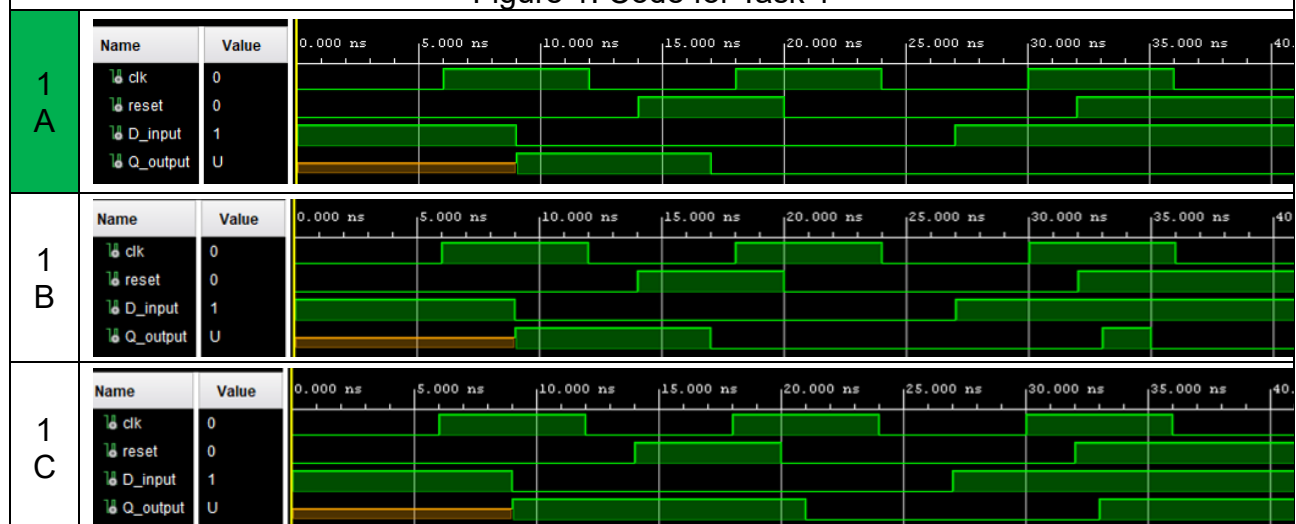
entity Task1_24 is
    port (
        clk      : in std_logic;
        reset     : in std_logic;
        D_input   : in std_logic;
        Q_output  : out std_logic
    );
end Task1_24;

architecture rtl of Task1_24 is
begin

    process(clk, reset) begin
        if(reset='1') then
            Q_output <= '0' after 3ns;
        elsif(clk'event and clk='1') then
            Q_output <= D_input after 3ns;
        end if;
    end process;

end rtl;
```

Figure 1: Code for Task 1



SOLUTION EXAMPLE:

The code reflects a single D-flipflop with asynchronous reset and delay from input to output of 3ns. Alternative 1C has synchronous reset, which is wrong. 1B has a short output pulse of 2ns, which is shorter than the delay through the flipflop. Since we are not using transport delay, pulses shorter than the delay will be suppressed, and consequently this alternative is also wrong. This leaves 1A as the correct answer.

Task 2: Matching block diagram and VHDL code

(Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the code and symbol definition in Figure 2, which block diagram matches the code, A, B, or C?

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Task2_24 is
  port( clk : in std_logic;
        a, b, c, d: in unsigned(7 downto 0);
        y : out unsigned(7 downto 0) );
end Task2_24;

architecture rtl of Task2_24 is
  signal sum_ab2: unsigned(7 downto 0);
  signal sum_cd2: unsigned(7 downto 0);
begin

  process(clk) is
    variable sum_ab1: unsigned(7 downto 0);
    variable sum_cd1: unsigned(7 downto 0);
  begin
    if(clk'event and clk = '1')
    then
      sum_ab1 := a + b;
      sum_ab2 <= sum_ab1;
      sum_cd2 <= sum_cd1;
      sum_cd1 := c + d;
    end if;
  end process;

  y <= sum_ab2 + sum_cd2;

end rtl;
```

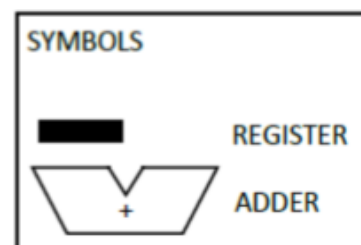
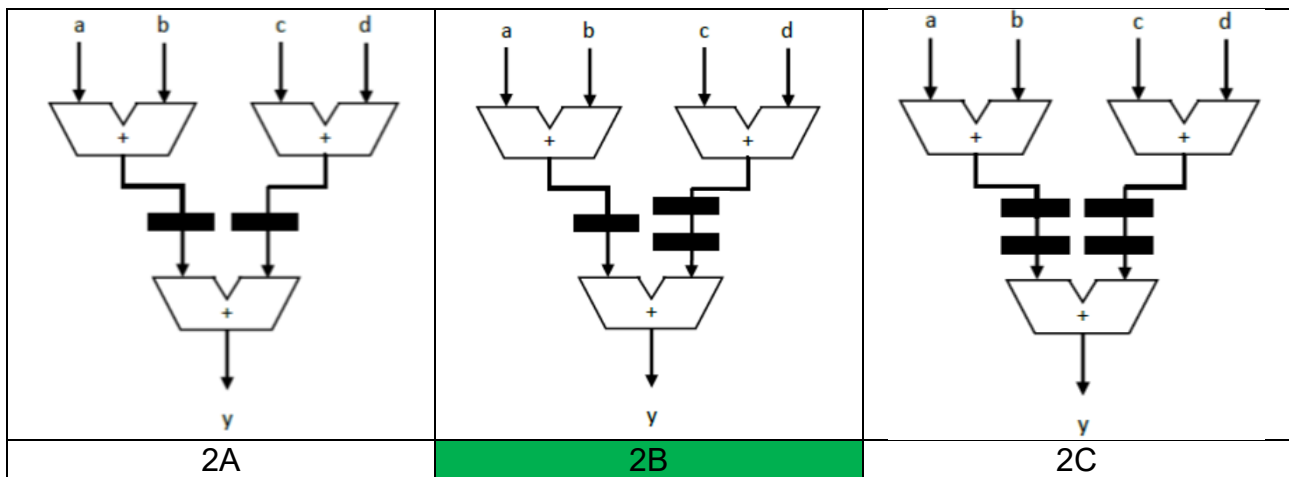


Figure 2: Code and symbol definition for Task 2



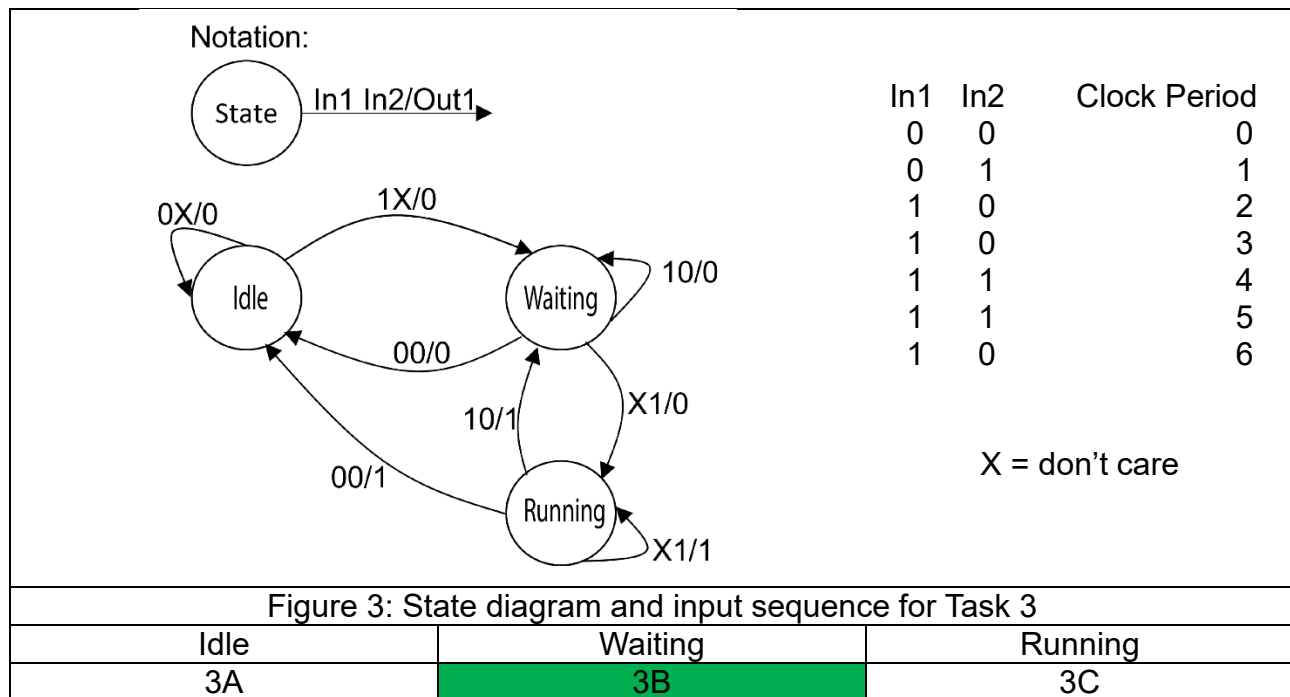
SOLUTION EXAMPLE:

We have a mix of variables and signals in the code. Signal assignments will in clocked processes like the one in Figure 2 give rise to registers due to the delta delay. Results on variables are immediately available, which means that when `sum_ab2` is calculated, `sum_ab1` is already available. Since statements inside a process is executed sequentially, the new `sum_d1` is not available when `sum_cd2` is calculated. Hence, a separate register is needed to store `sum_cd1` until the next clock flank, at which it is transferred to `sum_cd2`. This is reflected in Alternative 2B.

Task 3: State Machine

(Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the state diagram and the sequence of input signals in Figure 3. Assume that `In1` and `In2` change value on the negative flank of a clock, while the state machine changes state on the positive flank of the same clock. All setup and hold times are complied with. Assume further that the state machine starts in state `Idle`. In what state do we end up after the final input values in Clock Period 6 has been evaluated by the state machine, A `Idle`, B `Waiting`, or C `Running`?



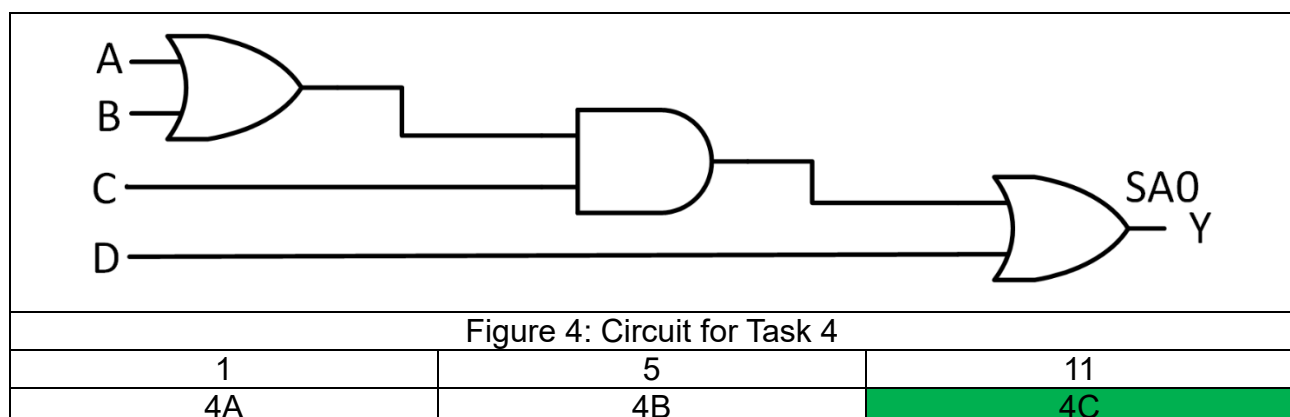
SOLUTION EXAMPLE:

The two first input patterns keep the state machine in state Idle. In clock period 2 we move to state Waiting. The input pattern in clock period 4 moves the state machine to state Running, from which it returns to state Waiting when the input pattern in clock period 6 is evaluated. Hence, we end up in Waiting after the final input value has been evaluated.

Task 4: Test pattern

(Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the circuit in Figure 4. How many different test patterns on the primary inputs A through E can detect the primary output Y SA0, A 1, B 5, or C 11?



SOLUTION EXAMPLE:

Since Y is a primary output, all we have to consider is controllability. All test patterns that give a 1 on Y will therefore detect Y SA0. With four input we have in total 16 possible test

patterns. As long as D is 1 it does not matter what A, B, and C is. This already give us 8 patterns that detect Y SA0. If D is 0, C must be 1 to have 1 on the upper input of the final OR-gate. At the same time, A or B must be 1. This gives us three additional test patterns, as also shown in the table below:

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Table 1: Solution example
Test patterns Task 4

Task 5: Result from synthesis

(Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the code in Figure 5. What type of circuit will result from synthesis of this process, A Combinatorial, B Sequential with at least one flip-flop, or C Sequential with at least one latch?

```
process(a,b,c,d,e) begin
  if(a='1') then
    y <= '1';
  elsif(b='0') then
    y <= '0';
  elsif(c=d) then
    y <= '1';
  elsif(e='0') then
    y <= '0';
  end if;
end process;
```

Figure 5: Code for Task 5

Combinatorial	Sequential with at least one flip-flop	Sequential with at least one latch
5A	5B	5C

SOLUTION EXAMPLE:

The process is not clocked, but since many of the input combinations are not covered and there is no final *else* in the if-statement, a previous value has to be stored in a latch if none of the *if / elsif* clauses match.

Task 6: Resulting signal value

(Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Assume that signal Y in Figure 6 is declared with the correct type. If this is a legal statement, what is the resulting value on Y, A the code line contains a syntax error, B '0', or C "1101"?

Y <= '1' & '1' & '0' & '1';		
Figure 6: Code line for Task 7		
Syntax error	'0'	"1101"
6A	6B	6C

SOLUTION EXAMPLE:

& is a concatenation operator. Hence, the four individual bit values are combined as in Alternative 6C.

Task 7: Timing calculation

(Correct answer 3 points, wrong answer -1 point, no answer 0 points)

You are going to implement the polynomial function $Y = X^3 - X^2 + X - 1$. Due to chip area constraints, you can use maximum two multiplier and two combined adder/subtractor modules. The multiplier has a critical path of 60ns while the combined adder/subtractor has a critical path of 40ns. If it is possible to implement, what is the minimum delay through a fully combinatorial circuit realizing the polynomial function, A Not possible, B 100ns, C 160ns, or D 200ns?

Polynomial function: $Y = X^3 - X^2 + X - 1$		Available resources: * * +/- +/-	
Figure 7: Function and resources for Task 7			
Not possible	100ns	160ns	200ns
7A	7B	7C	7D

SOLUTION EXAMPLE:

The function can be simplified as follows:

$$Y = X^3 - X^2 + X - 1 = X^2 (X - 1) + (X - 1) = (X^2 + 1) (X - 1)$$

This can be implemented as shown in Figure 7-A with a total critical path through two multipliers and one adder. That is 160ns as in Alternative 7C.

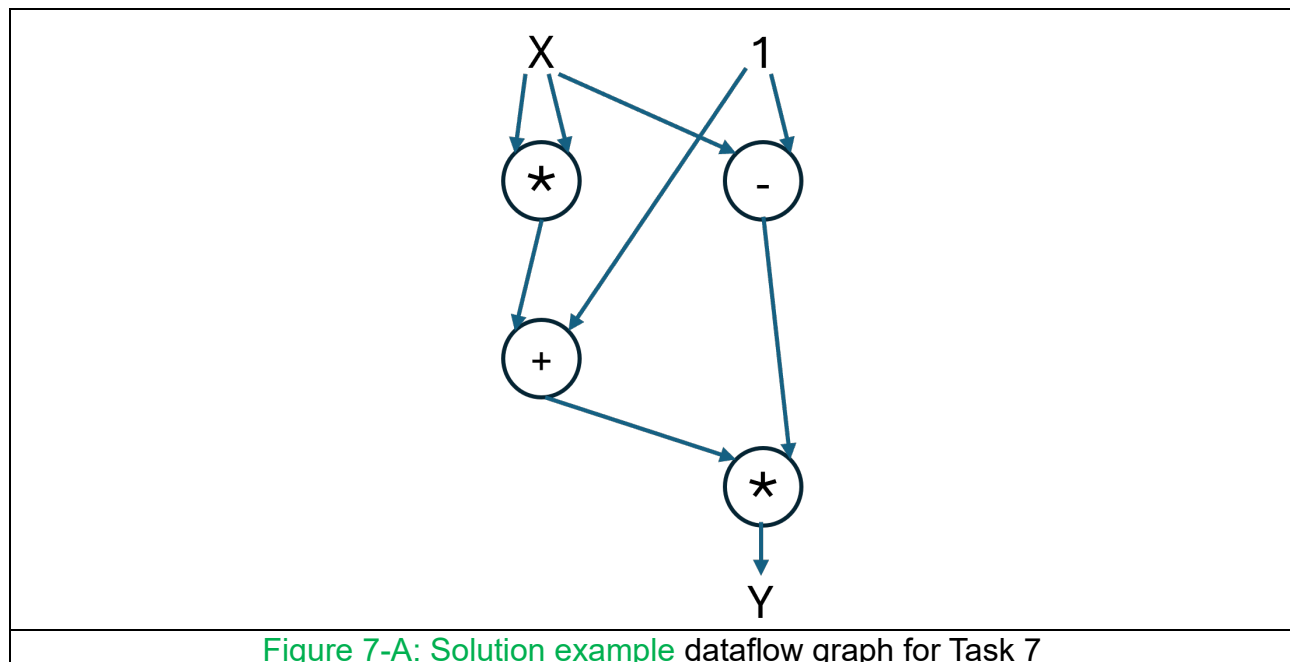


Figure 7-A: Solution example dataflow graph for Task 7

Task 8: Energy consumption

(Correct answer 3 points, wrong answer -1 point, no answer 0 points)

You need to estimate how much energy a chip consumes each time a multiplication is performed. You have the information in Figure 8 about the chip and multiplier module. Use the information you need to calculate if the estimated energy consumption is A 2.6pJ, B 14.4pJ, C 55.4pJ, or D 110.2pJ?

Supply voltage 1.2V Clock frequency: 150MHz Number of transistors in multiplier: 1 000 Average fan-out from each transistor output to input gates of other transistors: 2 Average transition activity on each transistor output during one multiplication (0->1 or 1->0): 0.8 Average transistor input gate capacitance: 10fF Average interconnect and parasitic capacitances of each transistor output: 5fF Note: $f = 10^{-15}$, $p = 10^{-12}$			
Figure 8: Information about chip and multiplier in Task 8			
2.6pJ	14.4pJ	55.4pJ	110.2pJ
8A	8B	8C	8D

SOLUTION EXAMPLE:

To calculate the energy for a given multiplication, we can use $E = N C V^2$, where N is the total number of transitions from 0 -> 1, C is the average capacitive load charged during each transition, and V is the supply voltage.

With the information in Figure 8, N can be found as the number of transistors multiplied with the average transition activity divided by two (since only transitions $0 \rightarrow 1$ counts). This gives $N = 400$.

The average load capacitance of each transistor is the average fan-out multiplied with the gate capacitance plus the average interconnect and parasitic capacitance. This gives $C = 25\text{fF}$.

This gives $E = 400 \times 25 \times 10^{-15} \times 1.2^2 = 14.4\text{pJ}$ as in Alternative 8B. Note that the clock frequency does not influence this energy consumption.

Task 9: True/false statements

(Correct answer 3 points, wrong answer -1 point, no answer 0 points)

How many of the following three statements are true, A 0, B 1, C 2, or D 3?

- i. PPA requirements capture the functional aspects of a design.
- ii. The order of VHDL code lines written directly inside an architecture influences the simulation or synthesis result.
- iii. If two sources drive the same `std_logic` signal to '1' and 'W', respectively, the resulting signal value will be 'W'.

0 are true	1 is true	2 are true	3 are true
9A	9B	9C	9D

SOLUTION EXAMPLE:

PPA captures non-functional requirements like Power, Performance, and Area, so i is false. All code lines written directly inside an architecture (not inside a process) is assumed to run in parallel. The order is hence not important and ii is false. 'W' is weak unknown in `std_logic`, so a '1' will be the resulting value after resolution and iii is false. Thus, alternative 9A is correct.

Task 10: True/false statements

(Correct answer 3 points, wrong answer -1 point, no answer 0 points)

How many of the following three statements are true, A 0, B 1, C 2, or D 3?

- i. VHDL allows overloading, which means that we can have several different implementations of, e.g., the `+` operator.
- ii. The statement `A <= B <= C;` contains two relational operators.
- iii. You should not have more than one wait statement in a process if it is going to be synthesizable.

0 are true	1 is true	2 are true	3 are true
10A	10B	10C	10D

SOLUTION EXAMPLE:

VHDL allows overloading, so i is true. The first $\leq$ in the statement is an assignment of the result to A. Hence there is only one relational operator and ii is false. A wait statement inside a process can be used as an alternative to a sensitivity list. For synthesis, only one wait statement (or sensitivity list) is allowed, so iii is true. Thus, alternative 10C is correct.

Task 11: True/false statements

(Correct answer 3 points, wrong answer -1 point, no answer 0 points)

How many of the following three statements are true, A 0, B 1, C 2, or D 3?

- i. Simulation in VHDL using signals is normally faster than simulation using variables.
- ii. If you instantiate components in your VHDL code, you will in addition use a separate component configuration.
- iii. In VHDL, all procedures require a return statement.

0 are true	1 is true	2 are true	3 are true
11A	11B	11C	11D

SOLUTION EXAMPLE:

Unlike signals, variables do not use an event queue during simulation, and is therefore faster. Hence i is false. You need to configure an instantiated component, so ii is true. Functions require a return statement, not processes, so iii is false. Thus, alternative 11B is correct.

Task 12: True/false statements

(Correct answer 3 points, wrong answer -1 point, no answer 0 points)

How many of the following three statements are true, A 0, B 1, C 2, or D 3?

- i. In VHDL, assertions can be used to report error conditions during simulation.
- ii. In advanced testbench design, verbosity control can be used to limit the size of the input data set used during simulation.
- iii. In standard cell libraries you can find cells consisting of multiple gates in order to reduce cell area and interconnect requirements.

0 are true	1 is true	2 are true	3 are true
12A	12B	12C	12D

SOLUTION EXAMPLE:

Assertions can be used to report error conditions during simulation, so i is true. Verbosity control can be used to limit the number of messages that are reported by a testbench during simulation, so ii is false. A standard cell library typically contains optimized multi-gate cells, so iii is true. Thus, alternative 12C is correct.

REMEMBER that for task 7 through 12 you can include a short explanation of your answer in addition to using the table in the Appendix. See Page 1 for details.

Task 13: Reduced energy consumption (11 points)

Figure 9 shows a small design where the two inputs a and b are added to produce the sum y. There are registers on all inputs and outputs. The di_valid signal is high when a and b are valid and the do_valid signal is high when the output y is valid.

Suggest a change to the use of di_valid and do_valid in order to reduce the energy consumption of the design. Explain why your suggestion is a good idea and why the energy consumption is reduced. Include a new version of Figure 9 as part of your answer.

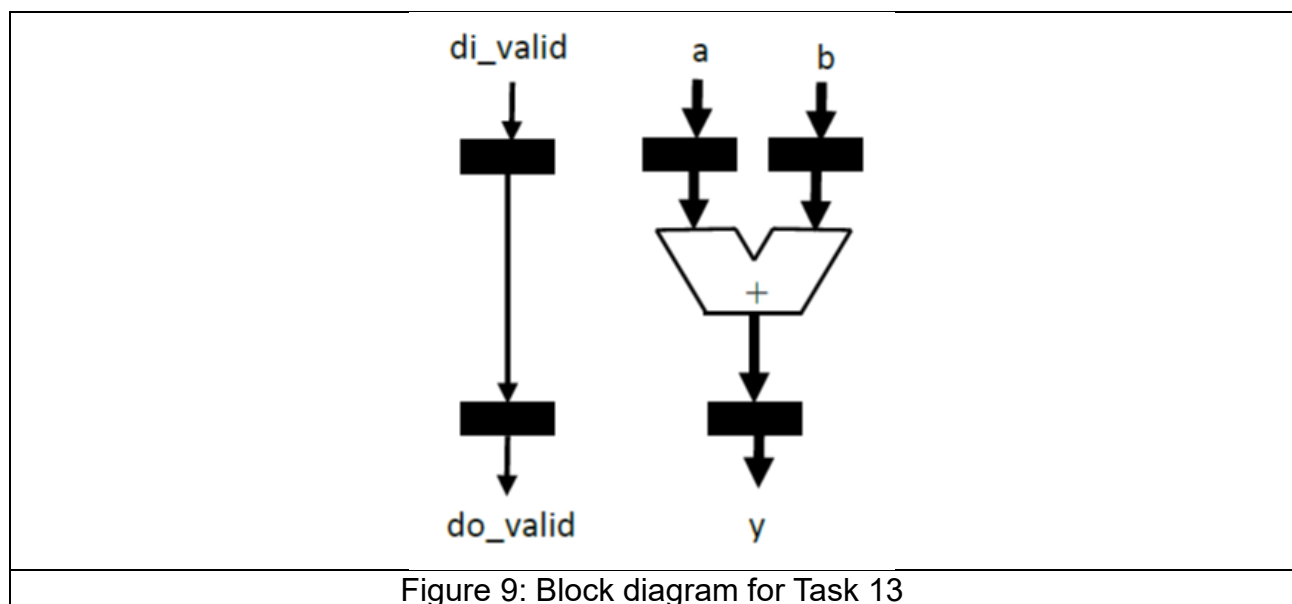


Figure 9: Block diagram for Task 13

SOLUTION EXAMPLE:

To reduce the switching activity in the adder and in the logic having y as input, you only want to update the registers when valid data is present. di_valid can thus be used as an enable signal for the input registers, while the internal valid signal (which is also the input to the do_valid register) can be used as an enable signal for the output registers. The valid signals can either be used directly if you have a load enable input on the register, or it can be used to perform clock gating of the registers. Given that the registers are wide enough to make clock gating beneficial, this will reduce the switching activity on the clock lines.

Reducing the switching activity reduces the energy (and power) consumption, as also exemplified in Taks 7, since it reduces the number of transitions N.

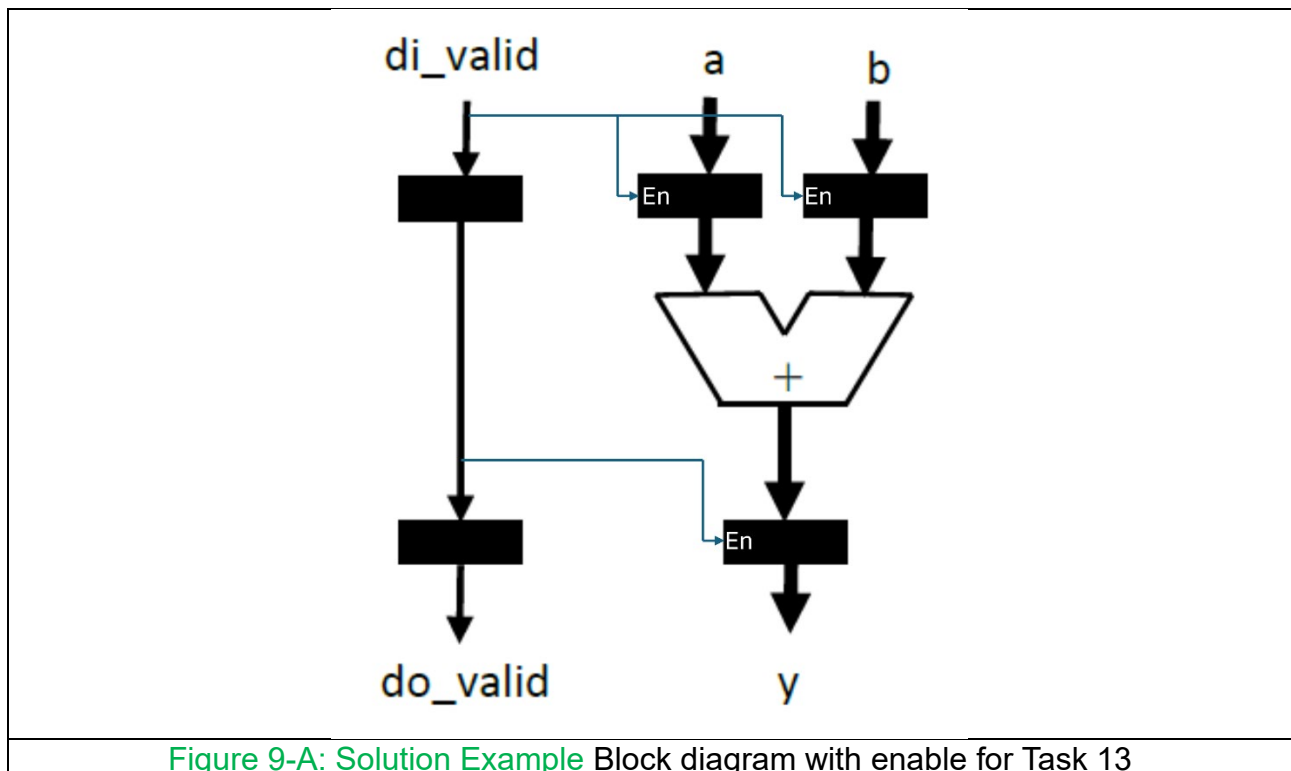


Figure 9-A: Solution Example Block diagram with enable for Task 13

Task 14: Coding of state machines

a) (Maximum 7 points)

Given the code for a state machine in Figure 10. Point out at least two errors in the code. With errors we here mean that the code does not follow standard practice for state machine coding, which will result in errors during compilation and/or that the synthesis not necessarily will be as expected. Explain with a few words why these are errors.

Do NOT consider that the state machine actually does not perform a sensible function.

SOLUTION EXAMPLE:

There are three errors in the code. (1) There is no *when others* in the combinatorial process. This is against standard practice. (2) In the active state (after *when active*) there is no assignment of a value to `out_sig` if *else* is chosen. This will result in an unwanted inferred latch. (3) The synchronous process has asynchronous reset. The reset signal is, however, missing from the sensitivity list.

b) (Maximum 6 points)

Change the entity definition of the state machine in Figure 10 so that you through the use of a generic constant easily can instantiate state machines with any word width for the signals `in_sig` and `out_sig`. You are NOT supposed to consider how the architecture is to be implemented.

SOLUTION EXAMPLE:

See Figure 10-A.

```
entity Task14_24_FSM is
port(in_sig, clock, reset: in bit;
      out_sig: out bit);
end entity Task14_24_FSM;

architecture RTL of Task14_24_FSM is
type state is (idle, active);
signal curr_state, next_state : state;
begin

CombProc: process (in_sig, curr_state)
begin
    case (curr_state) is
        when idle =>
            out_sig <= '0';
            if in_sig = '0' then
                next_state <= idle;
            else
                next_state <= active;
            end if;
        when active =>
            if in_sig = '1' then
                out_sig <= '1';
                next_state <= active;
            else
                next_state <= idle;
            end if;
        end case;
    end process CombProc;

SynchProc: process (clock)
begin
    if reset = '1' then
        curr_state <= idle;
    elsif clock'event and clock='1' then
        curr_state <= next_state;
    end if;
end process SynchProc;

end architecture RTL;
```

Figure 10: VHDL code for state machine in Task 14.

```
entity Task14_24_FSM is
generic(width : positive);
port(in_sig : in bit_vector(width - 1 downto 0);
      clock, reset: in bit;
      out_sig: out bit_vector(width - 1 downto 0));
end entity Task14_24_FSM;
```

Figure 10-A: Solution Example entity with generic constant in Task 14.

Task 15: RSA design

a) (Maximum 12 points)

In the RSA term project, you had to implement an efficient modular multiplication. For example, your implementation would take as inputs two vectors to be multiplied, A and B, as well as the modulo vector N itself, and produce an output vector R. Explain how you did this based on the requirements of the RSA system, e.g., size of key and message, and possibly your own precalculated values.

Your explanations should be at a level of detail that would make it possible for one of the other students in the course to implement the module. Include block diagrams as you see fit. Note that it is only the modular multiplication you should explain, not the complete modular exponentiation.

SOLUTION EXAMPLE:

Blakely Multiplication

Blakely multiplication involves three key values: the key pair (private or public), modulus N, and the message M to perform modular multiplication. The process begins with an initial value for the message, R, and produces a result. The key values A, B, and N. If the most significant bit (MSB) of A is '1', B is added to R. At this point, B is within $N - 1$, and the remainder may have been shifted in the prior steps staying within $2N - 2$. Consequently, the total possibilities fall within $3N - 3$, requiring subtraction of N up to two times. Afterward, the remaining data and A are shifted left, repeating the process until all bits are processed.

Montgomery Multiplication

Montgomery multiplication begins by precomputing R^{-1} , the modular inverse of R, for transitioning values into and out of Montgomery space, which is based on powers of 2. All values are first converted into Montgomery form by multiplying with R modulo N. The multiplication itself proceeds in a bit-by-bit manner, processing the bits of the message iteratively. For each bit b_i of the multiplier, modular adjustments are computed as $q_i = (S_{i-1} + b_i A) \bmod 2$, ensuring divisibility by 2 for the next step. The intermediate result is then updated using $S_i = (S_{i-1} + q_i M + b_i A) / 2$, where division by 2 is efficiently handled as a binary

shift. The procedure continues until all bits are processed, yielding the result in Montgomery space. The result is converted back to standard form.

b) (Maximum 12 points)

Define an entity for your modular multiplication unit that is able to handle incoming and outgoing data according to the ready valid handshake protocol used in the term project. Write a VHDL testbench that instantiates the entity, applies minimum two sets of input data and checks that the result is correct. Each data sets shall contain one A and one B value. The key should be the same for all sets.

Note that you are NOT supposed to write VHDL code for the modular multiplication itself, only the testbench. It is possible to do this even if you have not answered Task 15 a).

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_moduMultUnit is
end tb_moduMultUnit;

architecture Behavioral of tb_moduMultUnit is

    component moduMultUnit
        Port (
            clk      : in  STD_LOGIC;
            reset    : in  STD_LOGIC;
            R        : in  STD_LOGIC_VECTOR(255 downto 0);
            A        : in  STD_LOGIC_VECTOR(255 downto 0);
            B        : in  STD_LOGIC_VECTOR(255 downto 0);
            modu     : in  STD_LOGIC_VECTOR(255 downto 0);
            valid_in : in  STD_LOGIC;
            ready_in : out STD_LOGIC;
            result   : out STD_LOGIC_VECTOR(255 downto 0);
            valid_out: out STD_LOGIC;
            ready_out: in  STD_LOGIC
        );
    end component;

    signal clk      : STD_LOGIC := '0';
    signal reset    : STD_LOGIC := '0';
    signal R        : STD_LOGIC_VECTOR(255 downto 0);
    signal A        : STD_LOGIC_VECTOR(255 downto 0);
    signal B        : STD_LOGIC_VECTOR(255 downto 0);
    signal modu     : STD_LOGIC_VECTOR(255 downto 0) := (others
        => '0');
    signal valid_in : STD_LOGIC := '0';
    signal ready_in : STD_LOGIC;
    signal result   : STD_LOGIC_VECTOR(255 downto 0);
    signal valid_out: STD_LOGIC;
    signal ready_out: STD_LOGIC := '0';

    constant clk_period : time := 10 ns;

begin

```

Figure 10-B: Solution example testbench code for Task 15 (part 1)

[illegible]

Figure 10-C: Solution example testbench code for Task 15 (part 2)

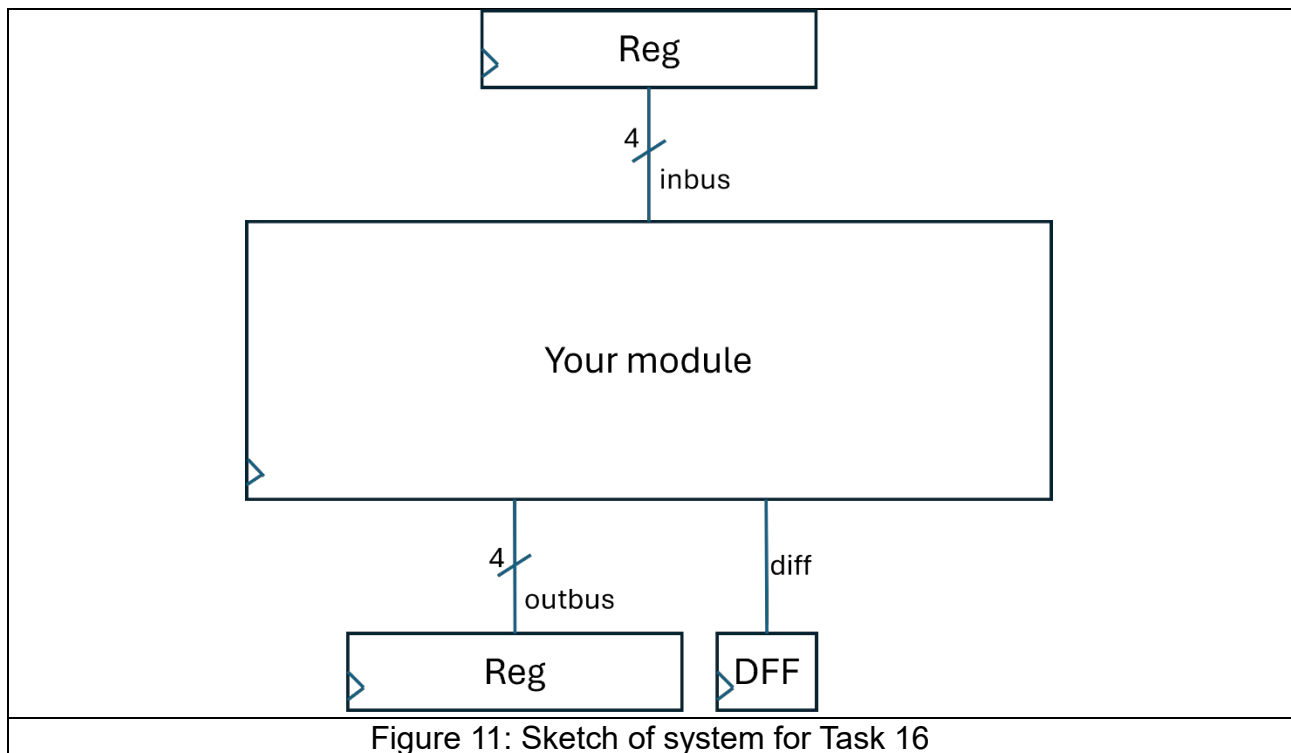
[illegible]

Figure 10-D: Solution example testbench code for Task 15 (part 3)

Task 16: VHDL Design

Together with the sketch in Figure 11, you have received the following module specification from your boss:

- Data arrives on a four-bit wide input bus, inbus of type std_logic_vector.
- The input bus is driven by a register which is clocked by the positive flank of the signal clk, but this is not part of your module (see Figure 11). You also have this clock signal available, and it is safe to assume that setup and hold times are complied with if you use the positive clock flank in your design.
- The module shall continuously compare the current value of each bit on the input bus with the value of the same bit during the previous clock period.
- If the two bits are equal, the corresponding bit of a four-bit std_logic_vector output bus, outbus, is to be set high. Otherwise it should be set low.
- The circuit shall, for example, compare the current inbus(2) and inbus(2) from the previous clock period and set outbus(2) high if they are equal and low if they are different. Similarly for the other bits of inbus and outbus.
- If three or more bits are different, a separate std_logic output signal, diff, shall be set high.
- The result on outbus and diff will be stored in registers clocked by the same positive clock flank, but this is not part of your module (see Figure 11).



a) (Maximum 15 points)

Draw a microarchitecture and write VHDL-code that satisfies the specification. Include the registers and D-flipflop outside your module in the microarchitecture drawing but not in your VHDL code.

Include any assumptions you have made during the design in your answer.

SOLUTION EXAMPLE:

We need to store inbus in order to compare successive values. In addition, we need a module that performs the comparison as well as a unit that detects the number of different bits. Figure 11-A shows a possible microarchitecture, assuming that the comparison and detection of different bits can be performed as combinatorial operations. The VHDL code in Figure 11-B shows possible ways of doing this. Compare is simply bitwise XNOR between the current and previous inbus. If three or four of the bits on outbus is low, diff should be high. This is the case for the five output vectors "0000", "0001", "0010", "0100", and "1000". This can easily be detected by a combinatorial select statement, as shown in the code.

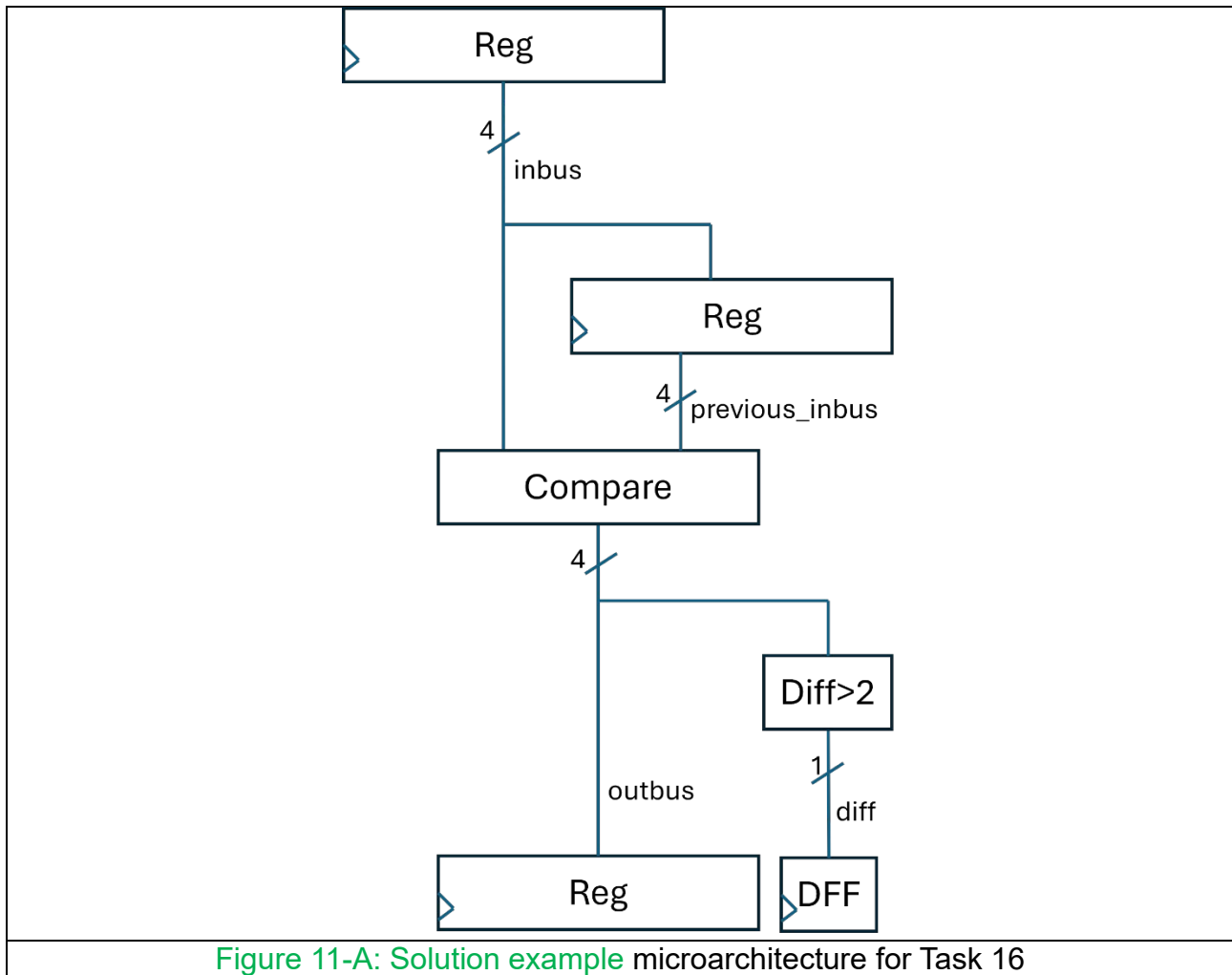


Figure 11-A: Solution example microarchitecture for Task 16

```
library ieee;
use ieee.std_logic_1164.all;

entity Task16_24 is
    port ( clk, reset : in std_logic;
          inbus : in std_logic_vector (3 downto 0);
          diff : out std_logic;
          outbus : out std_logic_vector (3 downto 0) );
end Task16_24;

architecture rtl of Task16_24 is
    signal previous_inbus : std_logic_vector (3 downto 0);
    signal compare_result : std_logic_vector (3 downto 0);
begin

    compare: process(clk, reset)
    begin
        if (reset = '1') then
            previous_inbus <= "0000";
        elsif (clk'event and clk='1') then
            previous_inbus <= inbus;
        end if;
    end process;

    compare_result <= previous_inbus XNOR inbus;
    outbus <= compare_result;

    with (compare_result) select diff <=
        '1' when "0000",
        '1' when "0001",
        '1' when "0010",
        '1' when "0100",
        '1' when "1000",
        '0' when others;

end rtl;
```

Figure 11-B: Solution example VHDL-code for Task 16

b) (Maximum 7 points)

Your boss wants to know what the performance of your implementation is. Make assumptions about the different basic components in your design, e.g., related to delay and timing of registers and combinatorial logic. Then calculate the clock frequency based on your assumptions and reflect on such topics as latency and throughput.

SOLUTION EXAMPLE:

We can assume the following delays in our circuit:

- Through register (T_{pd} from positive clock flank until register input is available on register output).
- Setup time on register storing the new outbus result (T_{setup} to ensure that register input do not change too close to a positive clock flank).
- Longest combinatorial delay from register output to register input (T_{comb}).

Let us assume that we have $T_{pd} = 2\text{ns}$ and $T_{setup} = 1\text{ns}$. T_{comb} consists of the delay through one two-input XNOR-gate and the diff-logic. The latter can, for instance, be realized using four inverters in parallel, followed by four three-input AND-gates parallel, followed by one four-input OR-gate (try putting the compare_result values in the select statement in Figure 11-B into a Karnaugh diagram). In total, this may give a critical path through the combinatorial logic of, e.g., approximately 2ns . This includes delay caused by capacitances on interconnect lines between gates and between registers and gates.

The total time that has to pass between two positive clock flanks is, hence, 5ns . This gives a maximum clock frequency of $F_{max} = 1/5\text{ns} = 200\text{MHz}$. Note that the assumed values are not necessarily realistic for modern chip technologies.

We need to have at least two consecutive data vectors on inbus before we can start generating outbus values. This can (possibly) be seen as a latency of two clock cycles. When the system is running, we have a new result every clock cycle, reflecting a throughput of one clock cycle.

If the system is implemented with sequential logic for Compare or Diff, we can get other clock frequencies and latency/throughput values.